SOFTWARE ENGINEERING MINI-PROJECT

CHAT APPLICATION

Real-Time Chat Application Using WebSockets

UNIT TEST REPORT

PROJECT DETAILS

Course: Software Engineering

Semester: 5th Semester

Section: K

Branch: Computer Science and Engineering (CSE)

University: PES University

Academic Year: 2026–2027

TEAM MEMBERS

Sl. No.	SRN	Name
1	PES2UG24CS625	Amoghaa R
2	PES2UG24CS664	Vaibhav R
3	PES2UG25CS817	P S Harshavardhan
4	PES2UG24CS648	Pramath Ganesh Hegde

UNIT TEST REPORT

1. Introduction

Unit testing is an important part of software testing that focuses on verifying individual components and functions of an application. It helps identify errors at an early stage and ensures that different parts of the application behave as expected.

For the Chat Application project, unit testing was performed using the Python Pytest framework. The testing focused on important server-side functionalities, including user authentication, messaging, group management, file sharing, security, presence notifications and administrative operations.

2. Objectives of Testing

The main objectives of unit testing were:

To verify the correctness of individual application functionalities.

To identify errors in authentication, messaging and other server-side operations.

To validate input handling and access control mechanisms.

To check the behaviour of the application under valid and invalid conditions.

To ensure that recent code modifications did not affect existing functionalities.

3. Testing Environment

The following environment was used to execute the unit tests:

Component	Details
Operating System	Windows 11
Programming Language	Python 3.14.2
Testing Framework	Pytest 9.2.0
Coverage Plugin	pytest-cov 7.1.0
Development Environment	Visual Studio Code
Project	Chat Application

4. Testing Methodology

The unit tests were implemented using Pytest. Individual test cases were written to verify different application functionalities.

Both positive and negative test scenarios were considered. Positive test cases checked whether valid operations were performed successfully, while negative test cases verified how the application handled invalid inputs, unauthorised access and unexpected conditions.

The test suite was executed using the following command:

python -m pytest tests -v --junitxml=tests/results/unit_test_results.xml

The verbose option displayed the execution status of individual test cases, while the JUnit XML option generated a test execution report.

5. Test Modules and Functionalities Covered

The test suite consisted of 12 test modules covering the following functionalities:

Test Module	Number of Tests
test_admin_coverage.py	14
test_auth_security.py	3
test_conversation_access_control.py	4
test_file_sharing.py	19
test_group_handlers.py	17
test_message_handlers_coverage.py	15
test_presence_notifications.py	15
test_rate_limit_and_status.py	13
test_secure_transport.py	4
test_shared_write_db.py	5
test_utility_coverage.py	21
test_ws_auth.py	11

Total	141

6. Test Execution Results

The complete unit test suite was executed successfully.

The execution produced the following results:

Result	Value
Total test cases executed	141
Test cases passed	141
Test cases failed	0
Test cases skipped	0
Execution time	49.71 seconds

All 141 test cases passed successfully.

One non-fatal deprecation warning was reported by the AnyIO pytest plugin. It did not affect the execution or results of the application tests.

7. Conclusion

The unit testing process was completed successfully for the Chat Application. A total of 141 test cases were executed, and all of them passed without any failures or skipped tests.

The testing covered the major server-side functionalities of the application, including authentication, messaging, file sharing, group management, security, presence notifications and administrative operations.

The results indicate that the tested components performed as expected under the selected test scenarios.